

Lecture: 0101

Latches, Flip-Flops & Timing – II

From Combinational to Sequential

How a circuit gains memory — and how a clock keeps that memory sane.

CSA222: Modern Computer Architecture

3. Flip-Flops

Sample the data at one precise instant — the clock edge — instead of a whole window. This is the reliable memory a CPU is built from.

Level-Triggered vs Edge-Triggered

The difference between 'listen the whole time' and 'listen for one instant'.

LEVEL-TRIGGERED (latch)

- Active during an entire clock LEVEL (high or low).
- Transparent — output tracks input the whole window.
- Simpler, fewer gates.
- Risky: data can slip through unintentionally.

EDGE-TRIGGERED (flip-flop)

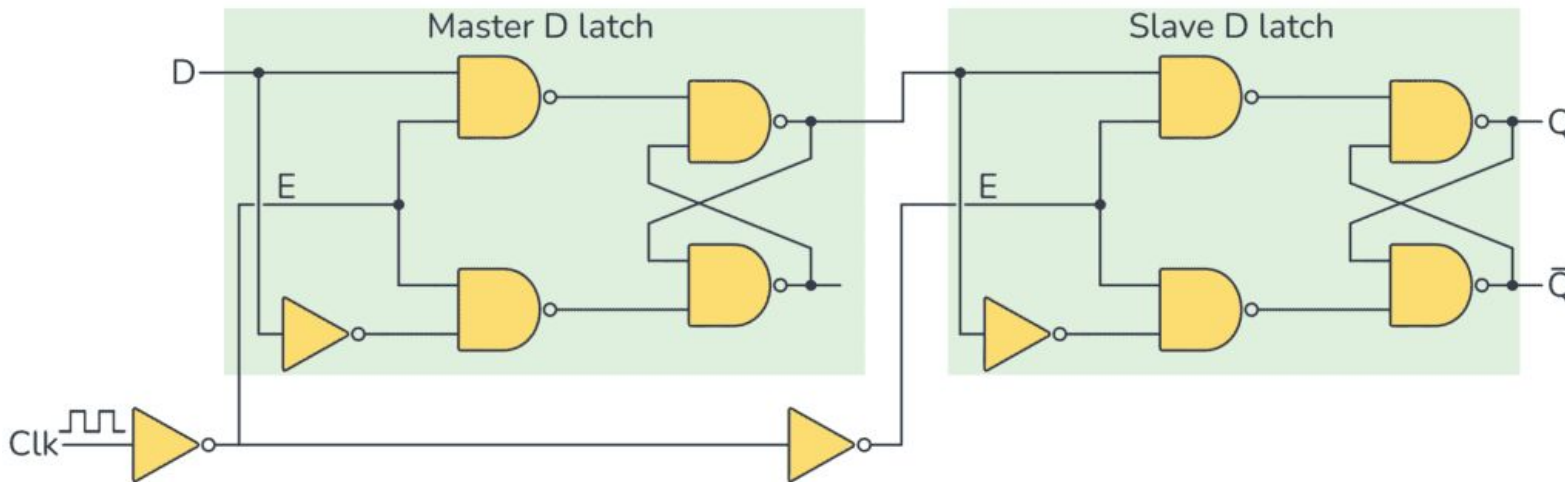
- Active only at the clock EDGE (rising or falling).
- Samples input at one instant, then locks.
- More gates, but predictable.
- The safe choice for pipelines & registers.

↑ rising edge

An edge is a MOMENT (0→1 transition), not a duration. That's why edge-triggering samples cleanly.

How Edge-Triggering Works: Master-Slave

Two latches in series on opposite clock phases — data can only advance one step per cycle.



THE HAND-OFF

The two latches are **NEVER** transparent at the same time.

When the master listens, the slave is locked — and vice versa.

So data can't race straight through. It's a two-stage airlock for one bit.

Master captures D while CLK is high; slave passes it on the falling edge. Result: one clean update per cycle.

Latch vs Flip-Flop

Same job — store a bit — but timing behaviour is completely different.

	LATCH	FLIP-FLOP
Triggered by	Clock LEVEL (high/low)	Clock EDGE (rising/falling)
Transparent?	Yes — while enabled	No — samples one instant
Gate count	Fewer (cheaper)	More ($\approx 2\times$ a latch)
Data races?	Possible — risky	Blocked — safe
Used for	Simple buffering	Registers, PC, pipelines

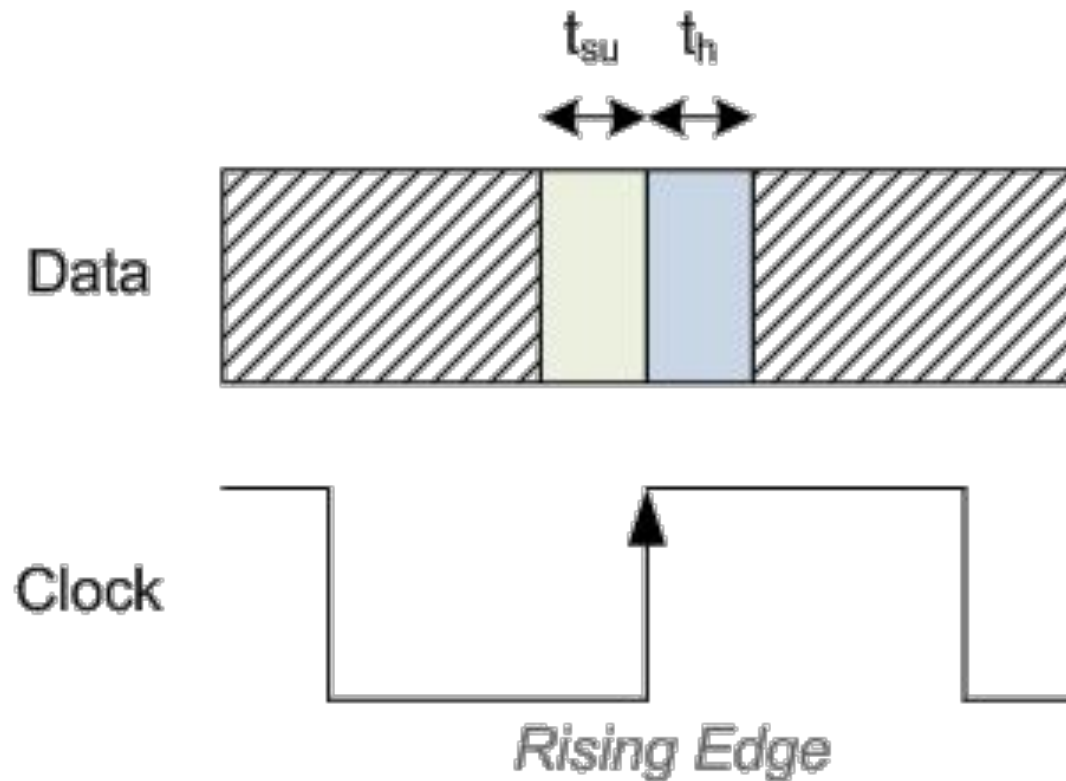
Rule of thumb: if a clock edge triggers it, it's a flip-flop. If a level does, it's a latch.

4. Timing

Flip-flops only work if data is stable around the clock edge. Break that rule and the bit becomes undefined.

Setup Time & Hold Time

Data must be STABLE in a window around the clock edge — before and after.



THE ANALOGY

Taking a photo of a moving object.

The subject must hold still just before AND just after the shutter clicks, or the picture blurs.

Same for a flip-flop: change D inside the setup/hold window and the captured bit is a blur — undefined.

Timing Violations

Break setup or hold, and the flip-flop's output is genuinely unpredictable.

SETUP VIOLATION

D changes too LATE — still moving when the edge arrives.

The new value didn't have time to settle before sampling.

HOLD VIOLATION

D changes too SOON — moves right after the edge.

The old value wasn't held long enough to be captured cleanly.

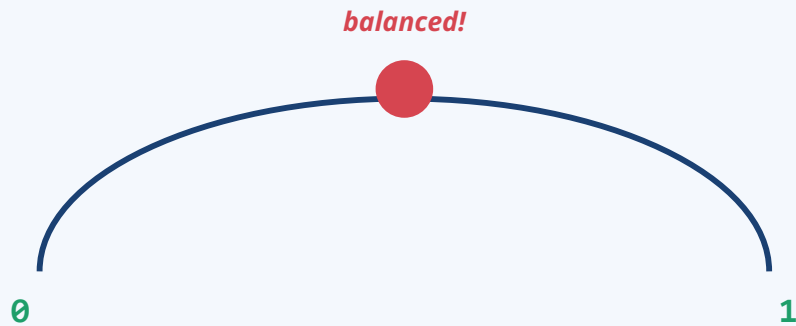
THE CONSEQUENCE

The output may settle to 0, settle to 1, or hover in between for a while — a condition called **METASTABILITY**. This is the deepest hazard in synchronous design, and it's next.

Metastability

A flip-flop caught between 0 and 1 — balanced on a knife's edge.

THE BALL-ON-A-HILL PICTURE



A valid bit is a ball resting in a valley (0 or 1). A timing violation places it on the peak — where it lingers, then falls to a side unpredictably, and after an unknown delay.

WHY IT'S DANGEROUS

The output is neither 0 nor 1 for an unpredictable time.

Downstream logic may read it as 0, another part as 1 — and the whole system's state diverges.

It's most common when crossing between two unrelated clocks (asynchronous inputs — buttons, other chips).

Living With Metastability

You can't eliminate it — but you can make it astronomically unlikely.



Can't be banned

For truly asynchronous inputs, metastability is physically unavoidable — no circuit prevents it 100%.



Synchronizer chains

Feed the signal through 2–3 flip-flops in series. Each gives the previous one time to settle before the next samples.



MTBF, not 'never'

Design targets a Mean Time Between Failures of thousands of years — practically never, but never truly zero.

The pro's mindset: you don't defeat metastability, you push its probability so low the chip will be obsolete before it ever happens.

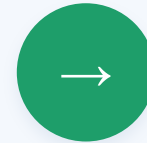
Timing in the Real World

Setup, hold, and the clock aren't theory — they set the speed limit of every chip.



Clock speed = the spec

'3.5 GHz' means the clock period is ~0.29 ns. All logic between two flip-flops must finish inside that window.



Critical path sets the max

The slowest logic path between flip-flops decides the fastest safe clock. Shorten it → clock faster.



Static Timing Analysis (STA)

EDA tools (Synopsys PrimeTime) check every path for setup/hold violations before a chip is ever built.

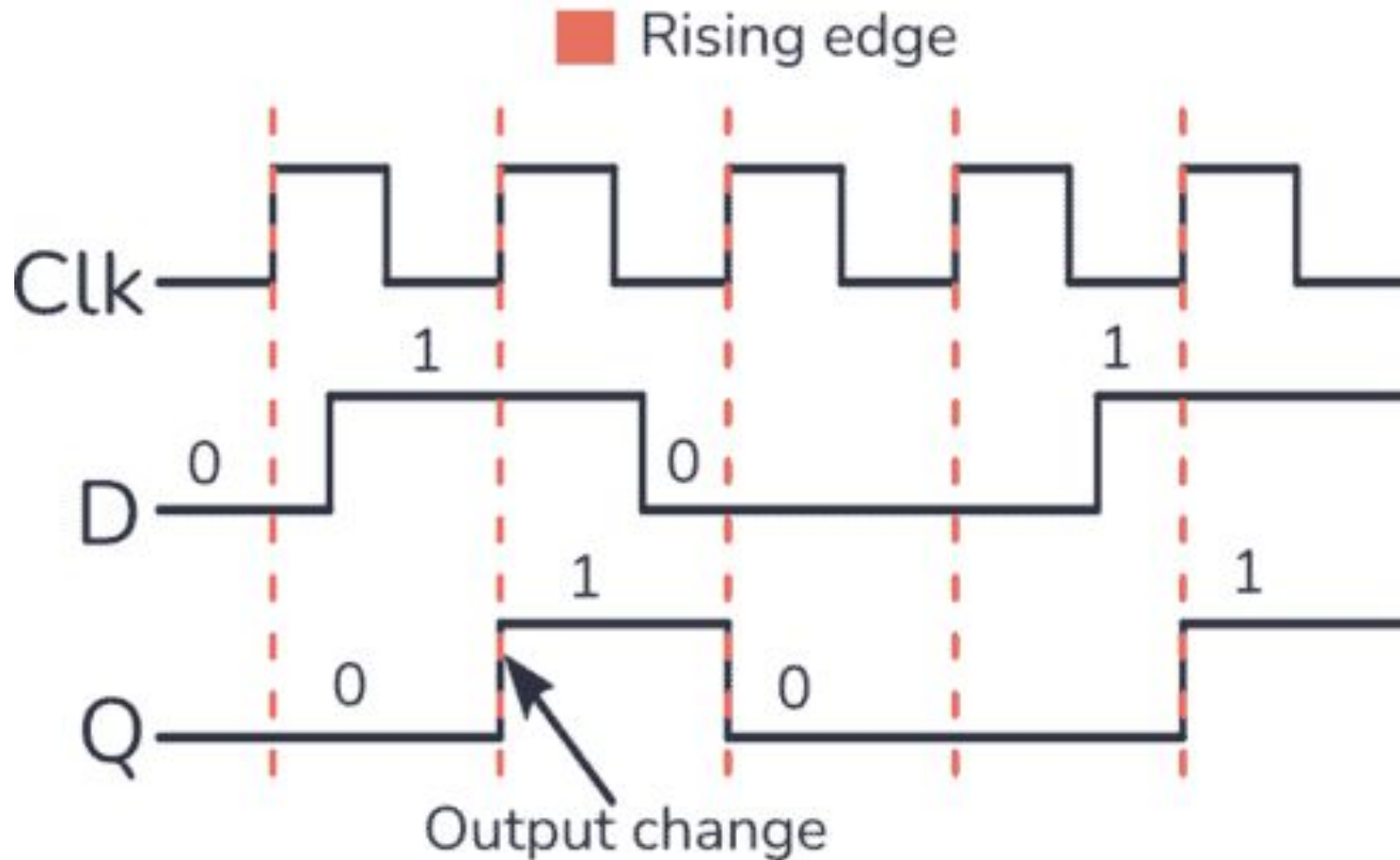


Overclocking = risking it

Pushing the clock past its rated speed shrinks timing margins — eventually setup violations corrupt data.

Reading a D Flip-Flop Waveform

Given CLK and D, trace Q — the classic exam and interview question.



Common Mistakes to Avoid

The slips that cost marks — and cause real timing bugs.



Latch vs flip-flop mix-up

A latch is level-triggered (transparent); a flip-flop is edge-triggered. The clock-input triangle marks a flip-flop.



Forgetting hold time

Setup gets the attention, but violating HOLD (D changes too soon after the edge) is just as fatal.



Allowing SR = 1,1

The forbidden state breaks $Q \neq Q'$. Real designs use a D latch to make it impossible.



Thinking metastability is a 'bug'

It's a physical certainty for async inputs — you manage its probability, you don't 'fix' it.



Ignoring the transparency window

A D latch copies D the whole time EN is high — not one clean value. Use a flip-flop for pipelines.



Zero-margin clocking

Running the clock right at the critical-path limit leaves no safety margin — temperature or voltage drift breaks it.

Key Takeaways

Five sentences — the birth of memory in a machine.

- 01 Feedback turns a stateless circuit into memory: an output routed back to its input can sustain a value.
- 02 An SR latch sets, resets, and holds a bit — but $S = R = 1$ is forbidden; the D latch engineers that danger away.
- 03 A latch is level-triggered (transparent while enabled); a flip-flop is edge-triggered (samples one instant).
- 04 The D flip-flop — built as master-slave — captures D on the clock edge and holds it, solving the transparency problem.
- 05 Data must be stable within the setup/hold window around the edge; violate it and you risk metastability, managed via synchronizer chains.

Practice & Homework

Trace, build, and reason about timing.

01

Latch Truth Tables

Complete the state tables for a NOR SR latch and a NAND SR latch. Mark the forbidden state in each.

02

Waveform Trace

Given a CLK and a D waveform, draw Q for both a D latch AND a D flip-flop. Explain the difference.

03

Timing Reasoning

A path has 0.28 ns delay; setup is 0.05 ns. What's the fastest safe clock frequency? Show your work.

EXIT TICKET

1. Why is $S = R = 1$ forbidden in an SR latch? What does the D latch do about it?
2. In one sentence, the difference between a latch and a flip-flop.
3. What is metastability, and why can't it be fully eliminated for asynchronous inputs?

Rapid-Fire Recap

Three questions — shout the answers.

Q1**A latch is triggered by a clock...****LEVEL****Q2****A flip-flop is triggered by a clock...****EDGE****Q3****Data unstable at the edge risks...****metastability**

All three instant? You're ready for the labs and L6 (Registers, Counters & Finite State Machines).

END OF L05

A loop, and a heartbeat.

NEXT — L06

Registers, Counters & Finite State Machines — chain flip-flops together and a circuit can count, remember words, and follow a plan.